

Master Androïde

Module MAOA

Maroua BAHRI

maroua.bahri@lip6.fr

Pierre FOUILHOUX

pierre.fouilhoux@lipn.univ-paris13.fr

2025-2026

Modalités

- Vous pouvez former des binômes
- Ce projet sera noté en fonction de trois aspects et comptera pour 20% de la note finale (+ 40% examen1 et 40% examen2)
- Une mini-soutenance/présentation de **10min** aura lieu le **28/01/2026 à 8h30**
- Soumission par email à:
 - maroua.bahri@lip6.fr
 - pierre.fouilhoux@lipn.univ-paris13.fr
- À rendre au plus tard le **22/01/2026 à 23h59**

Projet 1

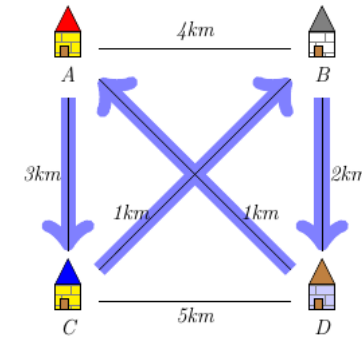
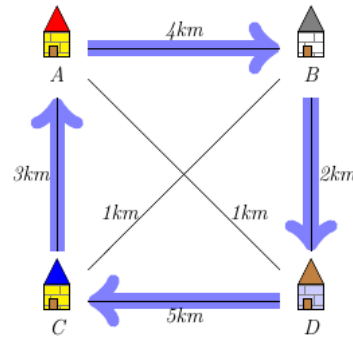
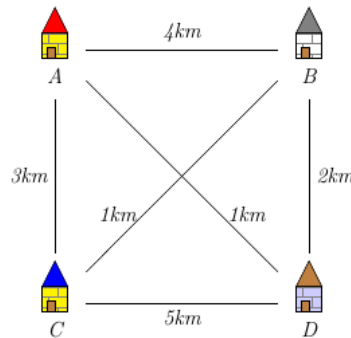
Traveling Thief Problem (TTP)

Traveling Thief Problem (TTP)

Combinaison de deux sous-problèmes bien connus:

- Problème du voyageur de commerce (TSP)

(Traveling Salesman Problem)



- Problème du sac à dos (KP)

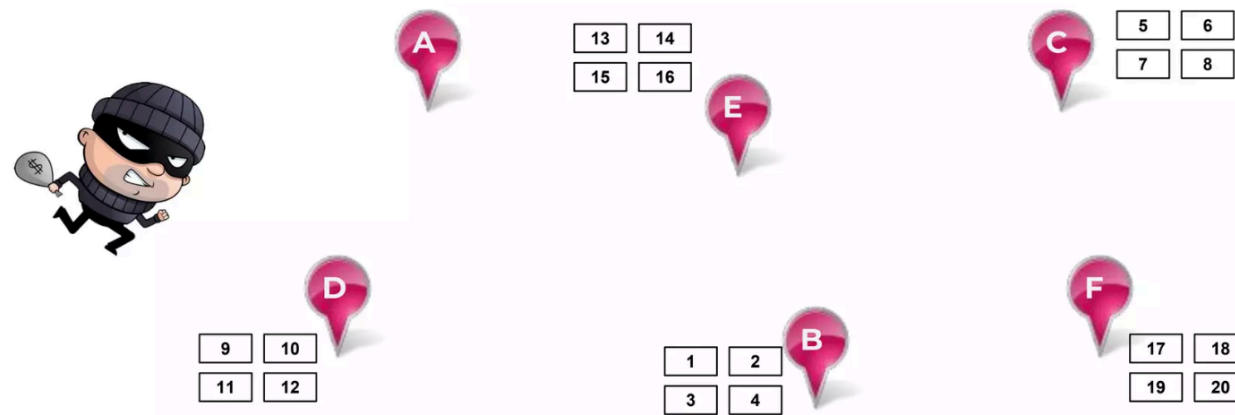
(Knapsack Problem)



Traveling Thief Problem (TTP)

Étant donné un ensemble d'objets distribués sur un ensemble de villes, un voleur doit visiter chacune des villes et décider quels objets voler.

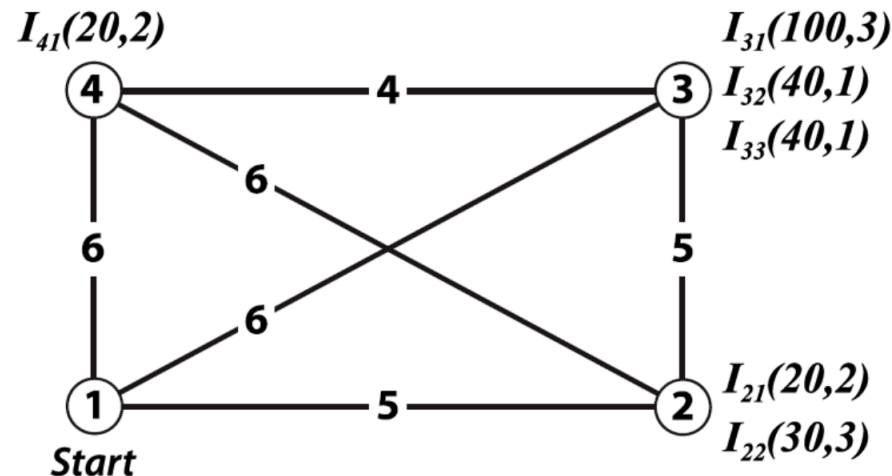
Quel est le meilleur chemin et le meilleur choix d'objets pour faire le meilleur profit ?



Applications: Logistique et transport, planification d'itinéraires pour la collecte de ressources, ...

Traveling Thief Problem (TTP)

- Le profit dépend de:
 - La valeur totale des objets volés
 - La longueur de la tournée
- La longueur de la tournée dépend :
 - Du chemin pris (TSP)
 - De la vitesse du voleur (déterminée par le poids total des objets volés dans le sac)



La tournée doit commencer et finir par la ville 1 (aucun objet n'existe dans la ville de départ/arrivée)

Traveling Thief Problem (TTP)

Données d'entrée/contraintes:

- Un ensemble de n villes $N = \{1..n\}$
 - Avec les distances $d_{i,j}$ entre chaque paire (i,j) de villes
- Un ensemble de m_i objets pour chaque ville: $M_i = \{1, \dots, m_i\}$, où chaque objet k dans la ville i est caractérisé par:
 - Un poids w_{ik} et sa valeur p_{ik}
- Un sac à dos avec une capacité maximale W
 - Sous la contrainte de sac à dos:
$$\sum_{i=1}^n \sum_{k=1}^{m_i} w_{ik} \leq W$$
- Un coût de location R du sac à dos (pour chaque unité de temps)
- Une vitesse de déplacement variable $v(w)$ qui dépend du poids w transporté
 - La vitesse diminue quand le poids augmente
 - La vitesse maximale est v_{max} et la vitesse minimale est v_{min}

Traveling Thief Problem (TTP)

- Le but est de trouver un tour passant par toutes les villes et la décision de prendre ou non un objet de chaque ville
 - Le parcours du voleur: tournée $\Pi = (x_1, \dots, x_n)$
 - Les objets à voler à chaque ville: plan de collecte (*packing plan*) $P = (y_{21}, \dots, y_{2m_2}, \dots, y_{nm_n})$
 $y_{ik} = 1$ si et seulement si l'objet k à la ville i est sélectionné
- Afin de maximiser le gain:

$$Z(\Pi, P) = B(P) - R * T(\Pi, P), \text{ où}$$

➤ $B(P)$ est le bénéfice, cad, le total des valeurs des objets choisis:

$$B(P) = \sum_{i=1}^n \sum_{k=1}^{m_i} p_{ik} y_{ik}$$

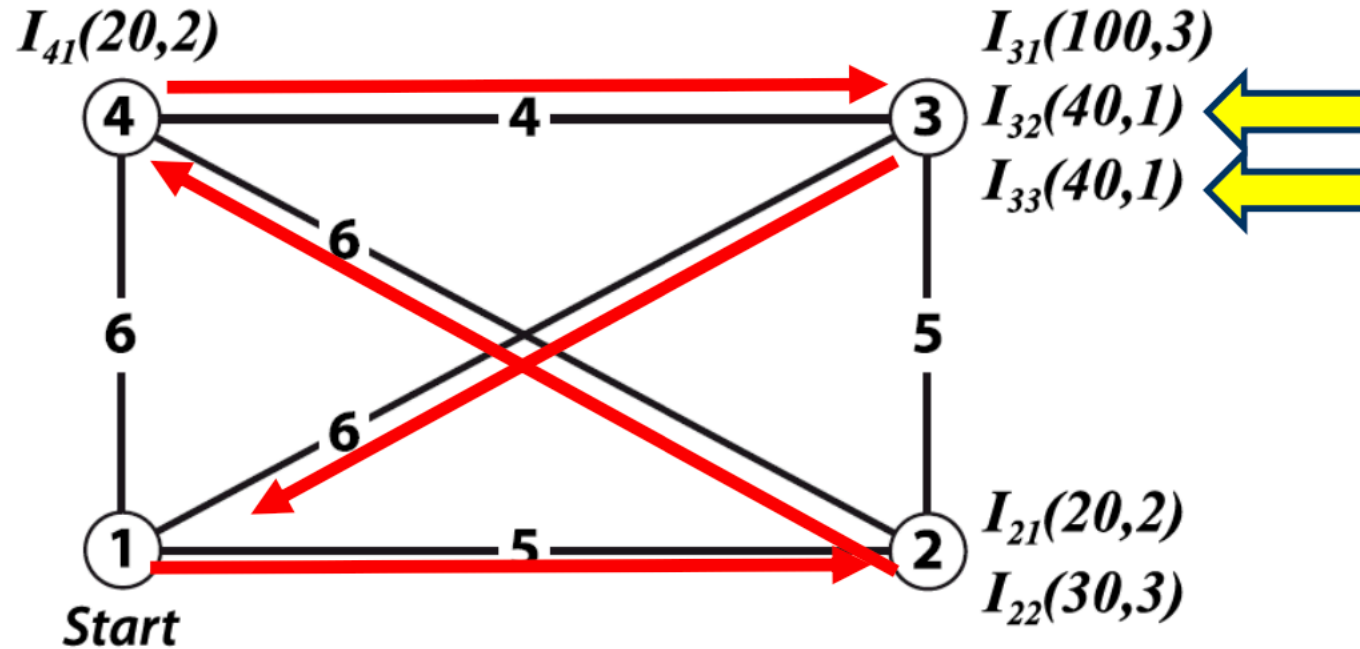
➤ $T(\Pi, P)$ est le temps que le voleur prend pour la tournée

$$T(\Pi, P) = \frac{d_{x_n x_1}}{v_{max} - v W_{x_n}} + \sum_{i=1}^{n-1} \frac{d_{x_i x_{i+1}}}{v_{max} - v W_{x_i}}$$

où $v = \frac{v_{max} - v_{min}}{W}$, et W_{x_i} représente le poids chargé dans le sac quand le voleur part de la ville i

Illustration de la fonction objective

- Entrée: $W = 3, v_{max} = 1, v_{min} = 0.1, R = 1$



- Profit de la tournée $\Pi = (1, 2, 4, 3)$ et du choix d'objets $P = (0, 0, 0, 0, 1, 1)$ est:

$$Z(\Pi, P) = 40 + 40 - 1 \cdot \left(15 + \frac{6}{1 - 0,3 \cdot 2} \right) = 80 - 30 = 50$$

TTP

Time and weight dependent TSP + contrainte de KP

Vous pouvez considérer deux formulations du problème:

1. Une première version du Time and weight dependent problem qui est appelé Cumulative TSP avec la contrainte de KP (KCTSP). Il y a une donnée supplémentaire:

- K coût de location du sac à dos (par km du transport d'un kg transporté)

- Le poids transporté à la ville x_i est alors $W_{x_i} = \sum_{k=1}^i \sum_{j=1}^{m_i} w_{kj} \cdot y_{kj}$

- Fonction **linéaire** qui ne considère pas l'aspect de vitesse (v_{min} , v_{max}) du TTP [3][4]

$$\sum_{i=1}^n \sum_{k=1}^{m_i} p_{ik} y_{ik} - K \left(d_{x_n x_1} W_{x_n} + \sum_{i=1}^{n-1} d_{x_i x_{i+1}} W_{x_i} \right)$$

2. Une deuxième version qui est le TTP avec une fonction objective Z qui est **non-linéaire**

Travail demandé

- L'idée est d'étudier les deux variantes en utilisant plusieurs méthodes de la littérature. Approfondissez au moins une des méthodes en s'appuyant sur la littérature
- Dans le cadre de ce projet, il est demandé:
 - Une visualisation des données et des solutions pour les deux versions
 - Une heuristique gloutonne pour TTP et KCTSP
 - Une heuristique itérative pour TTP et KCTSP
 - Une formulation compacte PLNE pour la version KCTSP résolu par GUROBI
 - Une comparaison des résultats obtenus pour les trois méthodes et les deux problèmes (en termes de qualité, temps de calcul, tailles résolues, ...)

Entrée: Il faut choisir une **limite** sur le nombre d'itérations ou le temps d'exécution pour chaque heuristique

Références

- [1] Polyakovskiy et al., A Comprehensive Benchmark Set and Heuristics for the Travelling Thief Problem. GECCO, 2014
- [2] Site web de CEC 2014 competition <http://cs.adelaide.edu.au/~optlog/CEC2014Comp/>
- [3] Abeledo et al., The Time Dependent Traveling Salesman Problem: Polyhedra and Branch-Cut-and-Price Algorithm. Experimental Algorithms. SEA 2010. LNCS, 6049 (2010).
- [4] Cordeau et al., Analysis and Branch-and-Cut Algorithm for the Time-Dependent Travelling Salesman Problem Transportation Science, 48-1 pp. 46-58 (2014).

Voir plus:

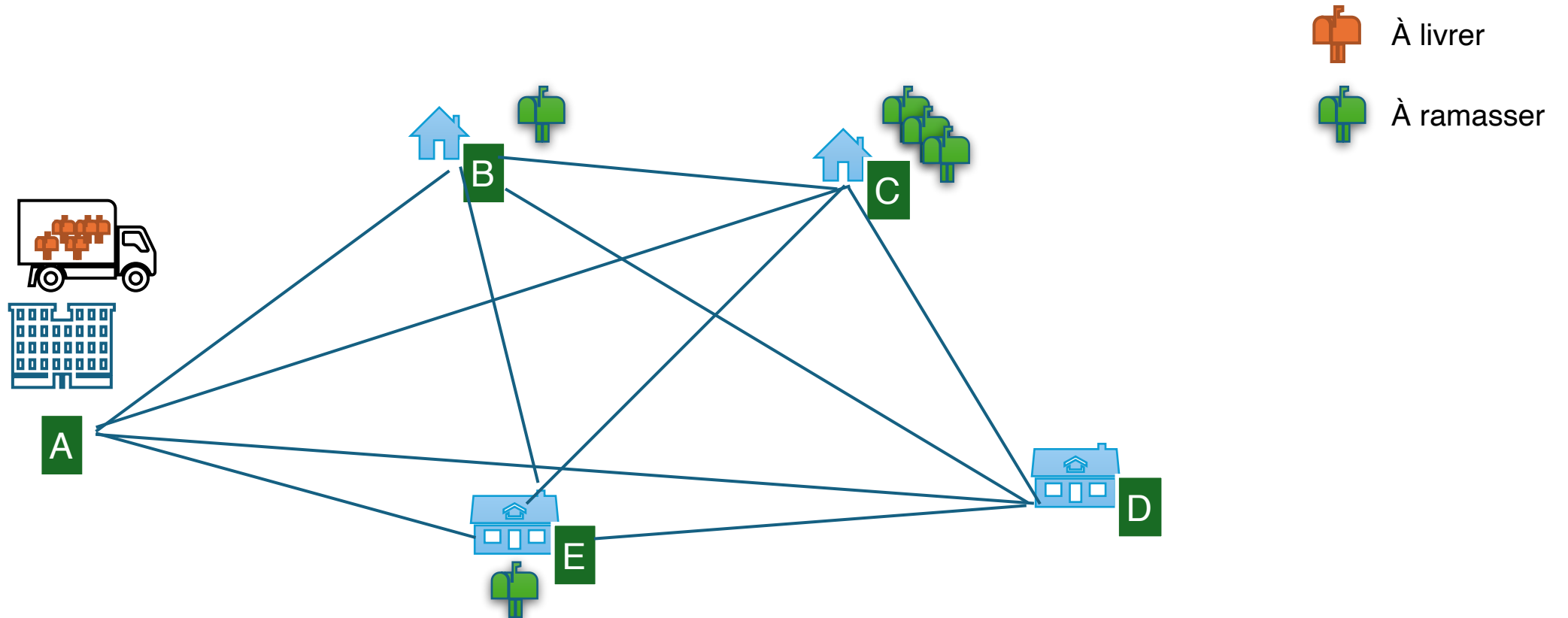
- [5] Bonyadi et al., The travelling thief problem: the first step in the transition from theoretical problems to realistic problems. IEEE CEC, 2013
- [6] Faulkner et al., Approximate Approaches to the Traveling Thief Problem, GECCO, 2015

Projet 2

Pick-and-Deliver TSP with Inventory Penalties (PD-TSP)

Pick and Deliver TSP (PD-TSP)

Combinaison de ramassages et livraisons



Pick and Deliver TSP (PD-TSP)

Un véhicule doit effectuer un ensemble de ramassages et de livraisons en visitant chaque ville une fois. On considère un tour TSP mais:

- À chaque ville, on peut livrer certains objets (déjà dans le véhicule)
- Et/ou ramasser de nouveaux objets des villes visitées

Le coût du déplacement dépend du poids transporté et peut être:

- Linéaire, ou
- Quadratique (coûts croissants de surcharge)

Le but est de trouver:

- Une tournée Π ,
- Un plan de chargement P (objets livrés & ramassés)

Applications: Courriers & services postaux: La Poste, DHL, UPS; Logistique médicale: collectes d'échantillons, livraison de médicaments; Recyclage: livraison de produits neufs + collecte de matériaux recyclables

Pick and Deliver TSP (PD-TSP)

Données d'entrée/contraintes:

- Un ensemble de n villes $N = \{1..n\}$
 - Avec les distances $d_{i,j}$ entre chaque paire (i, j) de villes
- Chaque ville i contient des objets déjà transportés au départ à livrer (**L**), chacun ayant un poids w_{ik}^L ($k \in L_i$)
 - Lorsque le véhicule atteint la ville i , il les livre, ce qui réduit son inventaire
- Chaque ville i contient des objets à ramasser (**R**), chacun ayant un profit p_{ik}^R et un poids w_{ik}^R ($k \in R_i$)
- À la sortie de chaque ville i , le poids devient:

$$W_i = W^0 - \sum_{k \in L_i} w_{ik}^L z_{ik} + \sum_{k \in R_i} w_{ik}^R y_{ik}$$

Où W^0 est l'inventaire initial; $z_{ik} = 1$ si l'objet à livrer est effectivement livré en i ; $y_{ik} = 1$ si l'objet est ramassé, 0 sinon

- La capacité du véhicule à chaque étape $W_i \leq W_{max} \forall i$

Pick and Deliver TSP (PD-TSP)

- Coût d'inventaire par arc: le coût de déplacement entre deux villes dépend du poids transporté. On considère deux variantes:

- Coût linéaire: $C_{ij}(W_i) = aW_i$
- Coût quadratique: $C_{ij}(W_i) = aW_i + bW_i^2$

Où a = coût marginal par unité de poids, b = pénalité croissante (e.g., surcharge)

- L'objectif exprime un compromis entre les profits de collecte et les coûts croissants liés au poids transporté au fil de la tournée
 - La tournée $\Pi = (x_1, \dots, x_n)$
 - Le plan de chargement $P = (z_{ik}, y_{ik})$
 - Il faut maximiser:

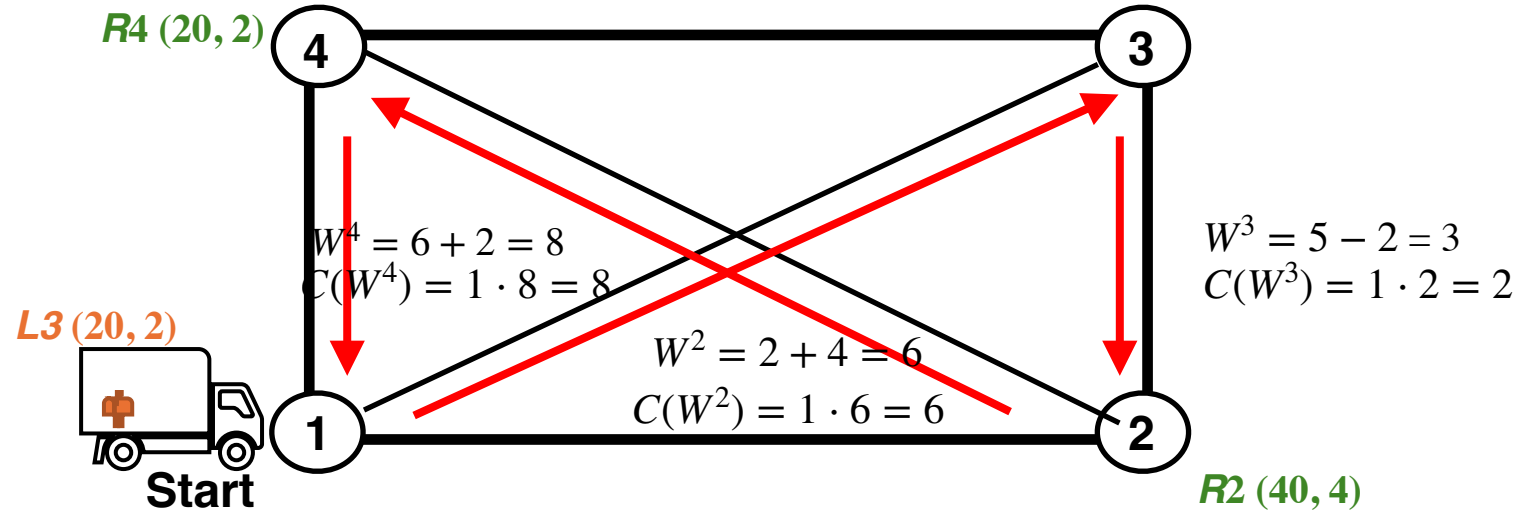
$$Z(\Pi, P) = \text{valeur ramassée} - \text{coût inventaire des arcs}$$

C'est à dire:

$$Z(\Pi, P) = \sum_{i,k} p_{ik}^R y_{ik} - \sum_{(i,j)} C_{ij}(W_i) x_{ij}$$

Illustration de la fonction objective

- Entrée: $W^0 = 5, a = 1, b = 0,2$



- La tournée $\Pi = (1, 3, 2, 4)$

$$Z(\Pi, P) = 60 - (10 + 2.8 + 12.2 + 20.8) = 39$$

Travail demandé

- L'idée est d'étudier les deux variantes en utilisant plusieurs méthodes de la littérature.
- Dans le cadre de ce projet, il est demandé:
 - Une visualisation des données et des solutions
 - Une heuristique gloutonne pour les deux versions basée sur des principes simples:
 - Livrer le plus tôt possible pour réduire le coût
 - Ramasser les objets les plus intéressants
 - Une heuristique itérative pour les deux versions
 - Une formulation PLNE résolue par GUROBI pour:
 - La version linéaire simple
 - Une comparaison des résultats obtenus pour les méthodes et les deux variantes linéaire vs quadratique (en termes de qualité, temps de calcul, ...)

Entrée: Il faut choisir une **limite** sur le nombre d'itérations ou le temps d'exécution pour chaque heuristique

Références

- [1] Veenstra et al., Pickup-and-Delivery TSP with Handling Costs. European Journal of Operational Research., 2017
- [2] Carrabs et al., Variable Neighborhood Search for the Pickup and Delivery Traveling Salesman Problem with LIFO Loading. INFORMS Journal on Computing., 2007
- [3] Malli et al., A computational study for the inventory routing problem. <https://arxiv.org/pdf/2007.14740>., 2020.

Travail demandé pour chaque projet

1. **Concevoir** des heuristiques pour chaque projet
 2. **Implémenter** ces heuristiques, les **évaluer**, et les **comparer** avec l'état de l'art (et entre elles)
 3. **Documenter** sous forme de rapport décrivant les solutions ainsi que les expérimentations réalisées avec une analyse critique des résultats
- Pour l'implémentation de vos solutions, utilisez le langage de programmation qui vous arrange
 - Rendre un code source complet et documenté des solutions (et les baselines utilisées) ainsi que les jeux de données utilisées pour les expérimentations
 - Le code et les données doivent être organisés dans une structure de répertoires claire
 - Options de partage recommandées: Un lien privé de dépôt GitHub, Drive, Dropbox, OneDrive, GitLab ou autre.
 - Fournir un lien de partage qui doit rester valide pendant la période d'évaluation, avec les droits d'accès en lecture accordés

- Questions?
- Quels sont les binômes ?